Master Diagnostic Checklist: Container Shipping & Orchestration (LO5)

To master container troubleshooting, you must categorize errors by the exact execution boundary where they occur: **CLI Runtime Flags**, **Image Build Architecture**, or **Orchestration Control Plane**.

Section 1: Podman & Runtime Execution Traps

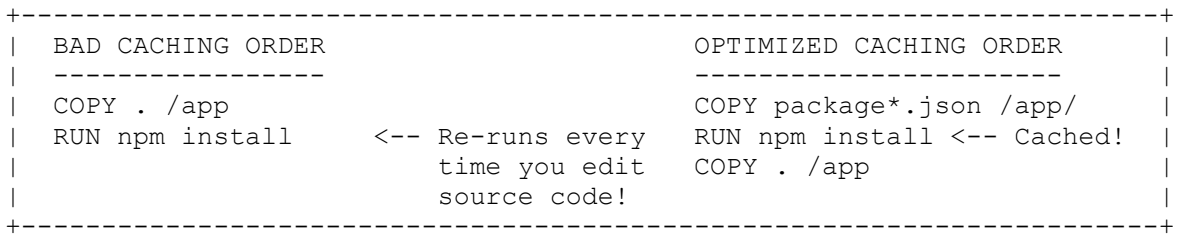
When a `podman run` command fails or behaves unexpectedly, isolate the flags before checking the image payload.

Trap Category	Flag / Command	Diagnostic Trigger	Root Cause & Resolution
Port Mappings	`-p <host>:<container>`	Connection refused from browser/curl on host port.	[Certain] Port order is strictly reversed. If Nginx listens on `80` and you want host port `8080`, syntax **must** be `-p 8080:80`.
Host Networking	`--network host -p 8080:80`	`-p` flag has no effect; app binds directly to host port `80`.	[Certain] Host networking merges container and host network namespaces. Port mapping flags are explicitly ignored by the runtime.
Background Ephemerals	`-d docker.io/library/busybox`	Container instantly enters `Exited (0)` state.	[Certain] Containers die the millisecond their primary foreground process (`PID 1`) exits. To keep basic utilities running, pass an active loop or block: `sleep infinity`.

Trap Category	Flag / Command	Diagnostic Trigger	Root Cause & Resolution
Detached Cleanup	`--rm -d <image> <failing-cmd>`	Container fails and vanishes before you can run `podman logs`.	[Certain] `--rm` destroys the container filesystem immediately upon process termination. Remove `--rm` during debugging so dead containers can be inspected.
Volume SELinux Denials	`-v ./data:/app/data`	`Permission denied` when reading/writing inside the container.	[Certain] On SELinux-enabled systems, host directories lack container access labels. Append `:Z` (private mount) or `:z` (shared mount) to relabel the directory: `-v ./data:/app/data:Z`.
Rootless DNS Isolation	`--name a` / `--name b`	`ping b` or `curl http://b` fails with `Name or service not known`.	[Certain] The default bridge network in rootless Podman does not provide embedded container name DNS resolution. Create a custom network (`podman network create devnet`) and attach both pods (`--network devnet`).

Section 2: Containerfile / Dockerfile Build Engineering

[Likely] Build failures and bloated images stem from fundamentally misunderstanding Linux filesystem layering and process signaling.



1. Layer Caching & Build Execution

- **Source Copy Placement:** [Certain] Never place `COPY . /workspace` before dependency installation (`RUN pip install`, `RUN npm install`, `RUN go mod download`). Source code edits invalidate subsequent layer caches.
- **Distro Package Managers:** [Certain] Always pair the correct package manager with your base distribution:
  - Debian / Ubuntu: `RUN apt-get update && apt-get install -y <pkg> && rm -rf /var/lib/apt/lists/*`
  - Alpine Linux: `RUN apk add --no-cache <pkg>`
  - RHEL / Fedora: `RUN dnf install -y <pkg> && dnf clean all`
- **Single-Layer Cleanup:** [Certain] Cleaning cache files in a *separate* `RUN` layer does not reduce image size. The underlying data remains stored in the previous read-only layer. Cleanup must occur in the exact same `RUN` command using `&&`.

2. Process Signaling & Security

- **Exec Form vs. Shell Form:** [Certain] Always use JSON array format for `CMD` and `ENTRYPOINT` (`CMD ["python3", "main.py"]`). Plain text strings (`CMD python3 main.py`) wrap your process in `/bin/sh -c`, which traps processes as child tasks and blocks `SIGTERM` signals, breaking graceful termination.
- **PATH Overrides:** [Certain] Setting `ENV PATH=/app/bin` overwrites system utilities. Always append or prepend: `ENV PATH="/app/bin:${PATH}"`.
- **User & Permissions Ordering:** [Certain] Switching to a non-root user (`USER appuser`) *before* copying root-owned files or installing system-wide packages causes `Permission denied` build breaks. Copy files, set ownership (`COPY --chown=appuser:appuser . .`), install system deps as root, and declare `USER` at the end of the file.

Section 3: Kubernetes Pod State Troubleshooting Matrix

[Certain] When a Kubernetes deployment breaks, the Pod status tells you precisely which subsystem failed.

Pod Status	Primary Diagnostic Command	Root Cause Analysis	Remediation Step
`Pending`	`kubectl describe pod <name>`	Look at the `Events` section at the bottom. Usually caused by: 1. Insufficient Node CPU/Memory capacity. 2. Missing PersistentVolumeClaim (`PVC`). 3. Unmatched `nodeSelector` or `taints`.	Lower resource requests, create the missing PVC, or adjust node affinity labels.
`ImagePullBackOff` / **`ErrImagePull`**	`kubectl describe pod <name>`	1. Image name typo or missing registry prefix.	Fix the image string in the manifest or attach the appropriate Docker registry secret to the ServiceAccount.

Pod Status	Primary Diagnostic Command	Root Cause Analysis	Remediation Step
		2. Non-existent tag (e.g., `:v2` instead of `:latest`). 3. Private registry missing an `imagePullSecrets` configuration.	
`CrashLoopBackOff`	`kubectl logs <name> --previous`	The application started, executed, and exited immediately with a non-zero status code (or code `0` if no daemon process exists).	Read the stack trace or startup logs from the dead container instance using `--previous`. Fix missing environment variables or fatal runtime errors.
`CreateContainerConfigError`	`kubectl describe pod <name>`	The kubelet cannot assemble the container environment because a referenced ConfigMap key or Secret key does not exist.	Verify keys using `kubectl get configmap <name> -o yaml` and correct typos in `configMapKeyRef` / `secretKeyRef`.

Pod Status	Primary Diagnostic Command	Root Cause Analysis	Remediation Step
`OOMKilled`	`kubectl get pod <name> -o yaml`	Look under `containerStatuses[*].lastState.terminated.reason`. The process exceeded its assigned `limits.memory` and was killed by the Linux kernel (`Exit Code 137`).	Increase `resources.limits.memory` in the deployment spec or fix application memory leaks.

Section 4: Live Troubleshooting & Actionable Diagnostic Commands

When an issue spans across networking and routing, execute this systematic verification sequence:

1. Diagnosing Broken Network Links (Ingress $\rightarrow$ Service $\rightarrow$ Pod)

If an application cannot reach a Service, check the exact chain of failure:

1. **Verify Pod Readiness:** Ensure target pods are labeled and Ready.

   Bash

   ```
   kubectl get pods --selector=app=mydb -o wide
   ```

2. **Verify Service Endpoints:** [Certain] If endpoints are empty (`<none>`), your Service `spec.selector` labels do not match your Pod `metadata.labels`.

   Bash

```
kubectl get endpoints <service-name>
```

3. **Verify Internal DNS & Port Routing:** Spin up an ephemeral debugging container to test cluster DNS and TCP reachability.

   Bash

   ```
   kubectl run tmp --rm -it --image=busybox -- /bin/sh
   # Inside ephemeral shell:
   nslookup <service-name>
   nc -zv <service-name> <port>
   ```

2. Deep-Dive Container Inspection Workflows

- **Inspect Distroless / Bare Images:** [Certain] If `kubectl exec -it <pod> -- /bin/sh` fails because the container has no shell, inject an ephemeral debug container into the pod's existing namespaces:

  Bash

  ```
  kubectl debug -it <pod-name> --image=busybox --target=<container-name>
  ```

- **Diagnose Stuck Rollouts:** When `Deployment` updates hang, query the rollout lifecycle status:

  Bash

  ```
  kubectl rollout status deployment/<name>
  kubectl describe deployment <name>
  ```

- **Verify Dry-Run Integrity Before Applying:** Catch YAML indentation errors, invalid API versions (`apps/v1` vs `core/v1`), and misnested port configurations without touching cluster state:

  Bash

  ```
  kubectl apply -f manifest.yaml --dry-run=server --validate=true
  ```